code; the same interface used to change access is used to update relationships. This makes ReBAC uniquely powerful: product managers, solutions architects, support engineers, and even customers themselves can safely participate in shaping access, all within guardrails set by a small set of clear, consistent rules.

In short, ReBAC allows ACME's Collaboration platform to scale access control without constant policy churn. A few well-defined rules are combined with a constantly changing graph of people and resources, resulting in clear, explainable, and flexible access decisions.

6.1.2 Mapping ReBAC onto the authorization reference architecture

Like ACLs and RBAC, relationship-based access control (ReBAC) fits cleanly into the authorization reference architecture described in chapter 4. What sets ReBAC apart is how each component functions, particularly the relationship between rules, data, and the user interface that modifies them.

Table 6.1 explains how each component of the architecture is usually implemented in a ReBAC system.

Table 6.1 Mapping ReBAC onto the authorization reference architecture. Each component is present in ReBAC systems, though often realized differently than in ACL or RBAC models.

Component	What It Does	How ReBAC Implements It
Policy Enforcement Point (PEP)	Intercepts access requests and enforces the decision	Often embedded in the application or gateway; enforces access based on PDP outcome
Policy Decision Point (PDP)	Determines whether the requested action is allowed	Authorization engine evaluates static rules against the current state of the relationship graph
Policy Administration Point (PAP)	Manages and stores roles, permissions, and assignments	Relationship graph updated via app UI (for example, share dialogs, team assignments)
Policy Information Point (PIP)	Supplies user and role data to the PDP at runtime	Combines relationship graph queries with data from the identity system or session context